

1. Flask App Scaffolding & Configuration

```
# src/app.py

from flask import Flask

from flask_cors import CORS

from routes.health import bp as health_bp
from routes.generate import bp as gen_bp
from routes.epics import bp as epics_bp
from routes.exports import bp as exp_bp


def create_app():

    app = Flask(__name__)

    CORS(app, origins=["http://localhost:3000"]) # Restrict to localhost


    # Register blueprints
    app.register_blueprint(health_bp)
    app.register_blueprint(gen_bp, url_prefix="/api/generate")
    app.register_blueprint(epics_bp, url_prefix="/api/epics")
    app.register_blueprint(exp_bp, url_prefix="/api/runs")


    return app


if __name__ == "__main__":
    app = create_app()
    app.run(port=5000, debug=True)
```

2. Configuration Management

```
# src/config.py
```

```
import os
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
class Config:
```

```
    # App
```

```
    APP_ENV = os.getenv("APP_ENV", "dev")
```

```
    PORT = int(os.getenv("PORT", 5000))
```

```
    # Jira
```

```
    JIRA_BASE_URL = os.getenv("JIRA_BASE_URL")
```

```
    JIRA_AUTH_METHOD = os.getenv("JIRA_AUTH_METHOD", "token")
```

```
    JIRA_EMAIL = os.getenv("JIRA_EMAIL")
```

```
    JIRA_API_TOKEN = os.getenv("JIRA_API_TOKEN")
```

```
    # Data Source
```

```
    DATA_SOURCE = os.getenv("DATA_SOURCE", "mock")
```

```
    EXPORT_DIR = os.getenv("EXPORT_DIR", "./out")
```

```
    # Limits
```

```
    MAX_EPICS_PER_REQUEST = int(os.getenv("MAX_EPICS_PER_REQUEST", 10))
```

```
    REQUEST_TIMEOUT_SEC = int(os.getenv("REQUEST_TIMEOUT_SEC", 20))
```

```
    RETRY_COUNT = int(os.getenv("RETRY_COUNT", 2))
```

3. Pydantic Models for Validation

```
# src/models/schemas.py
```

```
from pydantic import BaseModel, Field
```

```
from typing import List, Optional
```

```
from datetime import datetime
```

```
class EpicIn(BaseModel):
```

```
    epic_id: str = Field(..., min_length=1)
```

```
    title: str = Field(..., min_length=1)
```

```
    description: str = ""
```

```
class Constraints(BaseModel):
```

```
    stories_per_epic_min: int = Field(3, ge=1, le=10)
```

```
    stories_per_epic_max: int = Field(5, ge=1, le=10)
```

```
    tests_per_story_min: int = Field(2, ge=1, le=5)
```

```
    tests_per_story_max: int = Field(3, ge=1, le=5)
```

```
class GenerateRequest(BaseModel):
```

```
    project_name: str = Field(..., min_length=1)
```

```
    epics: List[EpicIn] = Field(..., min_items=1, max_items=10)
```

```
    constraints: Constraints = Constraints()
```

```
class JiraRequest(BaseModel):
```

```
    epic_keys: Optional[List[str]] = None
```

```
    jql: Optional[str] = None
```

```
    max_results: int = Field(5, ge=1, le=10)
```

```
class GenerateFromJiraRequest(BaseModel):  
    project_name: str = Field(..., min_length=1)  
    jira: JiraRequest  
    constraints: Constraints = Constraints()
```

```
class EpicOut(BaseModel):  
    epic_id: str  
    title: str  
    description: str
```

```
class RunEnvelope(BaseModel):  
    run_id: str  
    generated_at: datetime  
    output: dict
```

4. Jira API Client with Retry Logic

```
# src/services/jira_api.py

import requests

import base64

import time

from typing import List, Optional

from config import Config

import logging

logger = logging.getLogger(__name__)

class JiraClient:

    def __init__(self):

        self.base_url = Config.JIRA_BASE_URL

        self.auth = self._setup_auth()

        self.timeout = Config.REQUEST_TIMEOUT_SEC

        self.retry_count = Config.RETRY_COUNT

    def _setup_auth(self):

        if Config.JIRA_AUTH_METHOD == "token":

            credentials = f"{Config.JIRA_EMAIL}:{Config.JIRA_API_TOKEN}"

            encoded = base64.b64encode(credentials.encode()).decode()

            return {"Authorization": f"Basic {encoded}"}

        # Add other auth methods as needed

        return {}
```

```
def _make_request(self, method: str, endpoint: str, **kwargs):  
    url = f"{self.base_url}{endpoint}"  
  
    for attempt in range(self.retry_count + 1):  
        try:  
            response = requests.request(  
                method,  
                url,  
                headers=self.auth,  
                timeout=self.timeout,  
                **kwargs  
            )  
  
            if response.status_code == 429: # Rate limited  
                wait_time = 2 ** attempt  
                logger.warning(f"Rate limited, waiting {wait_time}s")  
                time.sleep(wait_time)  
                continue  
  
            response.raise_for_status()  
            return response.json()  
  
        except requests.exceptions.RequestException as e:  
            logger.error(f"Jira API attempt {attempt + 1} failed: {e}")  
            if attempt == self.retry_count:  
                raise
```

```
time.sleep(2 ** attempt) # Exponential backoff
```

```
def fetch_epic(self, key: str) -> Optional[dict]:
```

```
    try:
```

```
        data = self._make_request("GET", f"/rest/api/3/issue/{key}")
```

```
        # Validate it's an epic
```

```
        if data.get("fields", {}).get("issuetype", {}).get("name") != "Epic":
```

```
            logger.warning(f"Issue {key} is not an Epic")
```

```
            return None
```

```
        return {
```

```
            "epic_id": key,
```

```
            "title": data.get("fields", {}).get("summary", ""),
```

```
            "description": data.get("fields", {}).get("description", "")
```

```
        }
```

```
    except Exception as e:
```

```
        logger.error(f"Failed to fetch epic {key}: {e}")
```

```
        return None
```

```
def search_epics(self, jql: str, max_results: int = 5) -> List[dict]:
```

```
    try:
```

```
        params = {
```

```
            "jql": jql,
```

```
            "maxResults": max_results,
```

```
            "fields": "summary,description,issuetype"
```

```
}
```

```
data = self._make_request("GET", "/rest/api/3/search", params=params)
```

```
epics = []
```

```
for issue in data.get("issues", []):
```

```
    if issue.get("fields", {}).get("issuetype", {}).get("name") == "Epic":
```

```
        epics.append({
```

```
            "epic_id": issue["key"],
```

```
            "title": issue["fields"]["summary"],
```

```
            "description": issue["fields"].get("description", "")
```

```
        })
```

```
return epics
```

```
except Exception as e:
```

```
    logger.error(f"Failed to search epics: {e}")
```

```
return []
```

```
def fetch_epics(self, jira_payload: dict) -> List[dict]:
```

```
    if "epic_keys" in jira_payload and jira_payload["epic_keys"]:
```

```
        epics = []
```

```
        for key in jira_payload["epic_keys"]:
```

```
            epic = self.fetch_epic(key)
```

```
            if epic:
```

```
                epics.append(epic)
```

```
return epics
```

```
elif "jql" in jira_payload and jira_payload["jql"]:  
    max_results = jira_payload.get("max_results", 5)  
    return self.search_epics(jira_payload["jql"], max_results)  
  
return []
```

5. Mock Data Loader

```
# src/services/mock_loader.py
```

```
import json
```

```
import os
```

```
from typing import List, Optional
```

```
class MockLoader:
```

```
    def __init__(self):
```

```
        self.mock_file = "mock_data/sample_epics.json"
```

```
    def load_epics(self) -> List[dict]:
```

```
        try:
```

```
            with open(self.mock_file, 'r') as f:
```

```
                data = json.load(f)
```

```
            return data.get("epics", [])
```

```
        except Exception as e:
```

```
            print(f"Error loading mock data: {e}")
```

```
        return []
```

```
def get_epic(self, key: str) -> Optional[dict]:
```

```
    epics = self.load_epics()
```

```
    for epic in epics:
```

```
        if epic.get("epic_id") == key:
```

```
            return epic
```

```
    return None
```

```
def search_epics(self, project: str = None) -> List[dict]:
```

```
    epics = self.load_epics()
```

```
    if project:
```

```
        return [epic for epic in epics if project.lower() in epic.get("title", "").lower()]
```

```
    return epics
```

6. Health Check Route

```
# src/routes/health.py
```

```
from flask import Blueprint, jsonify
```

```
import time
```

```
bp = Blueprint("health", __name__)
```

```
start_time = time.time()
```

```
@bp.get("/health")
```

```
def health():
```

```
    return jsonify({
```

```
        "status": "ok",
```

```
    "uptime_sec": time.time() - start_time
})
```

7. Epic Discovery Route

```
# src/routes/epics.py
```

```
from flask import Blueprint, request, jsonify
```

```
from config import Config
```

```
from services.jira_api import JiraClient
```

```
from services.mock_loader import MockLoader
```

```
from models.schemas import EpicOut
```

```
bp = Blueprint("epics", __name__)
```

```
@bp.get("")
```

```
def list_epics():
```

```
    source = request.args.get("source", Config.DATA_SOURCE)
```

```
    if source == "jira":
```

```
        jql = request.args.get("jql", "issuetype=Epic")
```

```
        client = JiraClient()
```

```
        epics = client.search_epics(jql)
```

```
    else: # mock
```

```
        loader = MockLoader()
```

```
        epics = loader.load_epics()
```

```

    return jsonify([EpicOut(**epic).dict() for epic in epics])

@bp.get("/<key>")
def get_epic(key: str):
    if Config.DATA_SOURCE == "jira":
        client = JiraClient()
        epic = client.fetch_epic(key)
    else:
        loader = MockLoader()
        epic = loader.get_epic(key)

    if not epic:
        return jsonify({"error": "Epic not found"}), 404

    return jsonify(EpicOut(**epic).dict())

```

8. Environment Template

```

# .env

APP_ENV=dev

PORT=5000


# Jira Configuration

JIRA_BASE_URL=https://your-domain.atlassian.net

JIRA_AUTH_METHOD=token

JIRA_EMAIL=your-email@example.com

```

JIRA_API_TOKEN=your-api-token

Data Source

DATA_SOURCE=mock

EXPORT_DIR=./out

Limits

MAX_EPICS_PER_REQUEST=10

REQUEST_TIMEOUT_SEC=20

RETRY_COUNT=2

9. Requirements.txt

flask==2.3.3

flask-cors==4.0.0

pydantic==2.5.0

requests==2.31.0

python-dotenv==1.0.0

pandas==2.1.0

10. Basic Unit Tests

tests/test_jira_api.py

import pytest

from src.services.jira_api import JiraClient

from src.services.mock_loader import MockLoader

def test_mock_loader():

```
loader = MockLoader()
epics = loader.load_epics()
assert isinstance(epics, list)
```

```
def test_jira_client_auth():
    client = JiraClient()
    assert hasattr(client, 'auth')
```

```
def test_epic_validation():
    # Test epic validation logic
    pass
```